

Cho biết các kiến thức Toán mà sinh viên ngành Trí tuệ Nhân tạo được học trong năm 1, 2 nhưng không áp dụng trong các năm 3,4 khi qua giai đoạn chuyên ngành thuộc ngành Trí tuệ Nhân tạo

Câu hỏi của bạn rất hay vì nó chạm đúng "điểm đau" của nhiều sinh viên AI: học rất nhiều Toán ở năm 1–2 nhưng sang năm 3–4 lại ít thấy dùng trực tiếp. Tuy nhiên, cần nói rõ: **hầu hết các môn Toán đều có giá trị nền tảng**, chỉ là **mức độ sử dụng trực tiếp giảm đi** khi bạn chuyển sang học ứng dụng (machine learning, deep learning, NLP,...).

Dưới đây là phân tích rõ ràng theo từng nhóm kiến thức:

1. Những kiến thức Toán thường ít được áp dụng trực tiếp ở năm 3–4

(1) Giải tích 1 – đạo hàm, tích phân cơ bản

Học gì:

- Giới hạn, đạo hàm, tích phân
- Khảo sát hàm số
- Diện tích, thể tích

Thực tế AI:

-  Ít dùng trực tiếp (ít khi bạn tự tính tích phân tay)
-  Nhưng là nền tảng cho:
 - Gradient Descent
 - Backpropagation

 **Kết luận:** Không dùng "trực tiếp", nhưng là nền móng tư duy.

(2) Giải tích 2 – tích phân nhiều biến, chuỗi

Học gì:

- Tích phân kép, tích phân ba
- Chuỗi Taylor
- Hàm nhiều biến

Thực tế AI:

-  Gần như không dùng trực tiếp trong các môn như:
 - Lập trình AI

- Deep Learning ứng dụng
- ☒ Chỉ xuất hiện nếu học sâu:
 - Optimization nâng cao
 - Probabilistic models

👉 **Kết luận:** Thường bị "quên" nhanh nếu không đi theo hướng nghiên cứu.

📌 (3) Phương trình vi phân

Học gì:

- ODE (phương trình vi phân thường)
- Mô hình hóa hệ động lực

Thực tế AI:

- ☒ Hầu như không dùng trong AI ứng dụng phổ thông
- ☒ Có thể dùng nếu học:
 - Neural ODE
 - Control systems

👉 **Kết luận:** 90% sinh viên AI không dùng lại.

📌 (4) Hình học giải tích (phần cổ điển)

Học gì:

- Phương trình đường thẳng, mặt phẳng
- Hình học không gian

Thực tế AI:

- ☒ Ít dùng trực tiếp
- ☒ Nhưng tư duy vector rất quan trọng (chuyển sang Đại số tuyến tính)

👉 **Kết luận:** Phần "hình học thuần túy" ít dùng, nhưng vector thì cực kỳ quan trọng.

📌 (5) Một số phần của Xác suất – Thống kê cơ bản

Học gì:

- Xác suất cổ điển
- Biến ngẫu nhiên cơ bản
- Thống kê mô tả

Thực tế AI:

- ❌ Một số phần cơ bản ít dùng (ví dụ: bài toán xác suất cổ điển phức tạp)
- ✅ Nhưng phần quan trọng vẫn dùng:
 - Phân phối xác suất
 - Expectation
 - Bayesian

👉 **Kết luận:** Không phải bỏ hết – chỉ một phần ít dùng.

⚠️ 2. Những môn Toán tưởng “không dùng” nhưng thực ra rất quan trọng

Đây là chỗ nhiều sinh viên hiểu nhầm:

🔥 Đại số tuyến tính

- Vector, ma trận, eigenvalues
- 👉 DÙNG LIÊN TỤC trong AI

🔥 Xác suất – thống kê (phần nâng cao)

- Gaussian, likelihood, inference
- 👉 Cốt lõi của Machine Learning

🔥 Tối ưu (Optimization)

- Gradient Descent
 - 👉 Trái tim của Deep Learning
-

🎯 3. Vì sao bạn thấy “không dùng”?

Có 3 lý do chính:

1. Chuyển từ “tính tay” → “dùng thư viện”

- Năm 3–4: dùng TensorFlow, PyTorch
- Không còn giải toán thủ công

2. Tư duy > công thức

- Bạn dùng **concept**, không dùng bài tập kiểu sách giáo khoa

3. Học theo hướng ứng dụng

- Làm project > giải bài toán lý thuyết
-

✅ Tổng kết ngắn gọn

Ít dùng trực tiếp:

- Giải tích 1, 2 (phần tính tay)
- Phương trình vi phân
- Hình học giải tích cổ điển
- Một phần xác suất cơ bản

Vấn cực kỳ quan trọng (dù không thấy rõ):

- Đại số tuyến tính
- Xác suất – thống kê
- Tối ưu

💡 Nếu bạn muốn mình phân tích sâu hơn:

- Môn nào *thực sự cần để làm AI giỏi*
- Lộ trình học Toán tối ưu cho AI (bỏ cái gì, tập trung cái gì)
- Hoặc so sánh AI vs Data Science vs Software Engineering

Cứ hỏi, mình sẽ giải thích theo kiểu dễ hiểu nhất 👍

Kiến thức Toán nào cần cho việc học ngành Trí Tuệ Nhân Tạo trong năm 3,4 nhưng Sinh viên lại không được học trong năm 1, 2

Câu hỏi này rất "trúng vấn đề": không phải chỉ là *học thừa*, mà còn có **học thiếu**. Nhiều chương trình năm 1–2 dạy Toán khá "cổ điển", trong khi đến năm 3–4 học AI thì lại cần những kiến thức Toán **hiện đại và mang tính ứng dụng cao** mà sinh viên chưa từng được học bài bản.

Dưới đây là các nhóm kiến thức Toán rất cần cho AI ở năm 3–4 nhưng thường không được dạy (hoặc dạy rất ít) ở năm 1–2:

🧠 1. Toán Tối Ưu (Optimization) – “trái tim” của AI

Cần cho:

- Machine Learning

- Deep Learning (training model)

Kiến thức quan trọng:

- Gradient Descent, SGD, Adam
- Convex vs Non-convex optimization
- Learning rate, convergence

Vấn đề:

- Năm 1–2 chỉ học đạo hàm → **không đủ để hiểu tối ưu**
- Không được học:
 - Loss landscape
 - Overfitting liên quan đến tối ưu

👉 **Hệ quả:** Sinh viên dùng model nhưng không hiểu vì sao nó hội tụ hoặc “fail”.



2. Xác suất – Thống kê nâng cao (Applied Probability)

Cần cho:

- Naive Bayes
- Bayesian models
- Generative AI

Kiến thức thiếu:

- Bayes Theorem (ứng dụng sâu)
- Likelihood, Posterior
- KL Divergence, Entropy

Vấn đề:

- Năm 1–2 học xác suất kiểu:
 - Tung đồng xu
 - Bài toán tổ hợp
- 👉 Nhưng AI cần xác suất “liên tục + mô hình hóa”
-



3. Thống kê suy luận (Statistical Inference)

Cần cho:

- Đánh giá model
- A/B testing
- Data Science

Kiến thức thiếu:

- Hypothesis testing
- p-value, confidence interval
- Bias–variance tradeoff

👉 **Thực tế:** Nhiều sinh viên train model nhưng không biết:

- Kết quả có “ý nghĩa thống kê” không?

4. Đại số tuyến tính nâng cao (Applied Linear Algebra)

Cần cho:

- PCA (giảm chiều)
- Neural Networks
- Embedding (NLP)

Thiếu gì?

- SVD (Singular Value Decomposition)
- Eigen decomposition (ứng dụng)
- Vector space intuition (trực quan)

Vấn đề:

- Năm 1–2 học:
 - Giải hệ phương trình
- 👉 Nhưng không học cách áp dụng vào dữ liệu thực

5. Lý thuyết thông tin (Information Theory)

Cần cho:

- Decision Tree
- Deep Learning (loss function)
- NLP

Kiến thức thiếu:

- Entropy
- Cross-entropy
- Information gain

👉 Đây là thứ:

- Xuất hiện EVERYWHERE trong AI
- Nhưng gần như không dạy ở năm 1–2



6. Toán cho Deep Learning (rất thực dụng)

Cần cho:

- Neural Networks
- Computer Vision, NLP

Thiếu gì?

- Backpropagation (hiểu toán phía sau)
- Activation functions (ReLU, sigmoid)
- Chain rule trong thực tế



Sinh viên thường:

- Biết dùng PyTorch
- Nhưng không hiểu model đang làm gì 😊



Tổng kết ngắn gọn

! Những thứ “thiếu” lớn nhất:

- Optimization
- Xác suất ứng dụng (Bayesian)
- Statistical inference
- Linear algebra ứng dụng (SVD, PCA)
- Information theory
- Toán cho Deep Learning



Góc nhìn thực tế



Chương trình học thường:

- Năm 1–2: Toán “hàn lâm”
- Năm 3–4: AI “ứng dụng”



Khoảng trống nằm ở giữa:

- Không có cầu nối từ Toán → AI



Nếu bạn muốn đi nhanh hơn người khác

Bạn nên tự học thêm:

1. Optimization for ML

2. Probability for ML (không phải xác suất phổ thông)
 3. Linear Algebra ứng dụng
 4. Một chút Information Theory
-

Nếu bạn muốn, mình có thể:

- Lập lộ trình học Toán tối ưu riêng cho AI (cực practical)
- Hoặc chỉ ra môn nào nên bỏ qua – môn nào phải học sâu

Cứ nói mình biết mục tiêu của bạn (ML engineer, researcher, hay chỉ qua môn), mình sẽ hướng dẫn đúng hướng 👍

Giải pháp khắc phục các kiến thức Toán bị thiếu cho sinh viên ngành Trí tuệ Nhân tạo. Năm 3,4 cần kiến thức A nhưng không được học kiến thức A ở năm 1,2

Đây là một vấn đề **mang tính hệ thống** trong đào tạo AI: chương trình năm 1–2 không “chuẩn bị đúng thứ”, khiến sinh viên năm 3–4 phải **tự bù kiến thức** khi học chuyên ngành. Vì vậy, giải pháp không chỉ là “học thêm”, mà cần **cách học thông minh + chiến lược rõ ràng**.

Dưới đây là các giải pháp **thực tế, hiệu quả và có thể áp dụng ngay**:

1. Xây dựng “cầu nối” giữa Toán → AI (Bridge Learning)

👉 Vấn đề:

Sinh viên học Toán kiểu “giải bài tập”, nhưng AI cần “hiểu để dùng”.

👉 Giải pháp:

- Học Toán **gắn trực tiếp** với ứng dụng AI

Ví dụ:

- Đạo hàm → Gradient Descent
- Xác suất → Naive Bayes
- Ma trận → Neural Network



Cách học hiệu quả:

- Mỗi khi học 1 khái niệm Toán → hỏi ngay:

"Cái này dùng trong AI ở đâu?"



Đây là bước quan trọng nhất để "lấp khoảng trống".



2. Học Toán theo "Use-case" thay vì theo giáo trình



Thay vì:

- Học toàn bộ Optimization theory



Hãy học:

- Optimization cho Machine Learning



Ví dụ: Gradient Descent (cốt lõi AI)

$$y = f(x)$$

Bạn không cần học toàn bộ giải tích nâng cao, mà cần:

- Gradient là gì
- Vì sao đi ngược gradient thì giảm loss
- Learning rate ảnh hưởng thế nào



Tập trung vào:

- Hiểu bản chất > chứng minh



3. Micro-learning: học bù theo từng "lỗ hổng nhỏ"



Sai lầm phổ biến:

- "Thiếu Toán → học lại từ đầu"



Giải pháp tốt hơn:

- Chỉ học đúng phần đang thiếu để hiểu bài hiện tại

Ví dụ:

Khi học AI gặp

Thiếu gì

Học bù gì

Neural Network

Đạo hàm nhiều biến

Chain rule

Khi học AI gặp	Thiếu gì	Học bù gì
PCA	Eigenvalue	Eigenvector
Naive Bayes	Xác suất	Conditional probability

👉 Cách này giúp:

- Tiết kiệm thời gian
- Học đến đâu dùng đến đó



4. Học vòng lặp: Toán → Code → Toán

👉 Công thức học hiệu quả nhất:

Toán → Code → Quan sát → Quay lại Toán

Ví dụ:

1. Học Gradient Descent (Toán)
2. Code thử bằng Python
3. Quan sát loss giảm
4. Quay lại hiểu sâu hơn

👉 Kết quả:

- Hiểu sâu gấp 2–3 lần so với học lý thuyết



5. Tạo "Math Toolkit cho AI" (cá nhân hóa)

Bạn không cần toàn bộ Toán, chỉ cần một bộ "core":



Bộ Toán tối thiểu cho AI:

- Linear Algebra (vector, matrix, eigen)
- Probability (Bayes, distribution)
- Optimization (gradient descent)
- Statistics (evaluation model)
- Information theory (entropy)

👉 Mỗi phần chỉ cần:

- 20% lý thuyết → dùng được 80% bài toán



6. Giải pháp ở cấp chương trình đào tạo

Nếu nhìn ở mức trường/giảng viên:



Nên thay đổi:

- Thêm môn: "Mathematics for Machine Learning"
- Dạy:
 - PCA (thay vì chỉ ma trận)
 - Optimization (thay vì chỉ đạo hàm)
 - Xác suất ứng dụng (thay vì bài toán cổ điển)



Hoặc:

- Tích hợp Toán vào môn AI (teaching-in-context)



7. Chiến lược cá nhân cho sinh viên năm 3–4



Cách học nhanh nhất:

Bước 1: Xác định môn AI đang học

Bước 2: Liệt kê Toán cần thiết

Bước 3: Học đúng phần đó (YouTube / course ngắn)

Bước 4: Áp dụng ngay vào code



Ví dụ thực tế:

- Học Deep Learning →
 - Học thêm: gradient, chain rule, loss function
- Học NLP →
 - Học thêm: xác suất, entropy



Sai lầm cần tránh

- ❌ Học lại toàn bộ Toán từ đầu

- ❌ Học lý thuyết mà không code
 - ❌ Chỉ dùng library mà không hiểu
-

✅ Tổng kết

👉 Vấn đề không phải là:

“Không được học Toán”

👉 Mà là:

“Không học đúng Toán cần cho AI”

💡 Một câu rất quan trọng:

AI không cần bạn giỏi Toán hàn lâm — mà cần bạn hiểu Toán đủ để dùng.